

Combinatorial optimization layers in machine learning pipelines

ROAPA – ENPC

Guillaume Dalle

gdalle.github.io/

10.11.2025

Contents

Deep learning – setup	2
Deep learning – pipelines	8
Autodiff – basics	13
Autodiff – modes	20
Autodiff – implementation	41
CO layers – theory	49
CO layers – differentiation	58
CO layers – practice	75

Figures without attribution are borrowed from [1] or the works of our team

Deep learning – setup

We are given:

- A set of inputs $\mathcal{X}$
- A set of outputs $\mathcal{Y}$
- With an unknown distribution $p(x, y)$

We want to predict y given a new x .

We approximate $y = f_w(x)$ with $f_w \in \mathcal{F}$ a parametric model.

To measure our prediction error, we have a loss function $\ell(\hat{y})$.

Examples:

- Continuous prediction: $\ell(\hat{y}, y) = \|\hat{y} - y\|^2$
- Discrete prediction: $\ell(\hat{y}, y) = 1_{\hat{y} \neq y}$

Ideally, we want the parameters w that minimize the true risk:

$$\operatorname{argmin}_w \mathbb{E}_{(x,y) \sim p} [\ell(f_w(x), y)]$$

The true distribution p is unknown, but we have training data

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Deep learning – pipelines

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

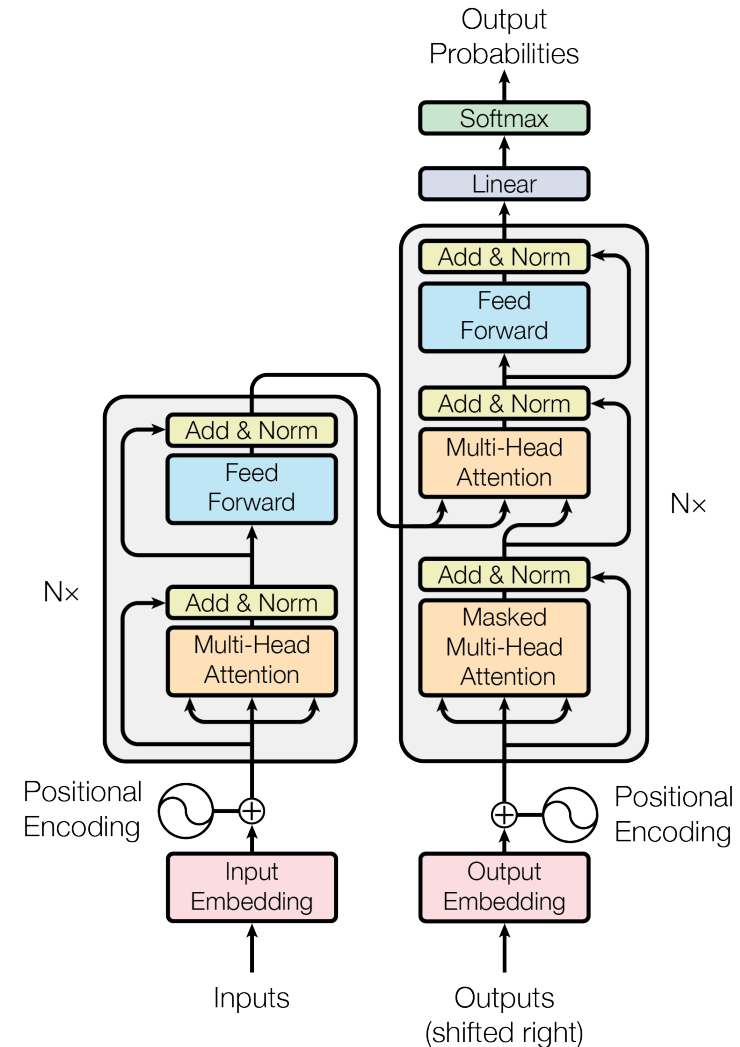
The need for automatic differentiation Deep learning – pipelines

Modern architectures are huge [2].

Don't want manual work when the model changes.

Automatic differentiation enables:

- easy experimentation
- modular code



The big picture

*Differentiable programming is a programming paradigm in which **complex computer programs** (including those with control flows and data structures) can be differentiated end-to-end automatically, enabling gradient-based **optimization of parameters** in the program.*

From the book [1] (see also [3])



Autodiff – basics

Definitions

Derivative = linear approximation of function f around point x :

$$f(x + v) = f(x) + \partial f(x)[v] + o(\|v\|)$$

Here $\partial f(x)[v]$ means “the linear map $\partial f(x)$ applied to v ”.

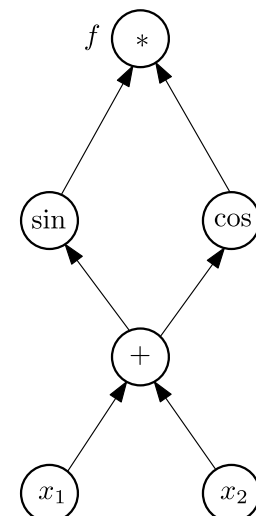
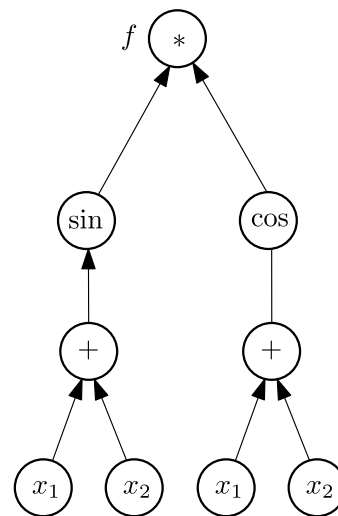
Plug the expression for $f(x)$ into a computer algebra system.

Gives an expression for $f'(x)$, possibly very long.

Expression trees are not great for computer programs:

- shared variables
- loops

Exercise [4]: expression tree & computational graph for $f(x) = \sin(x_1 + x_2) \cos(x_1 + x_2)$



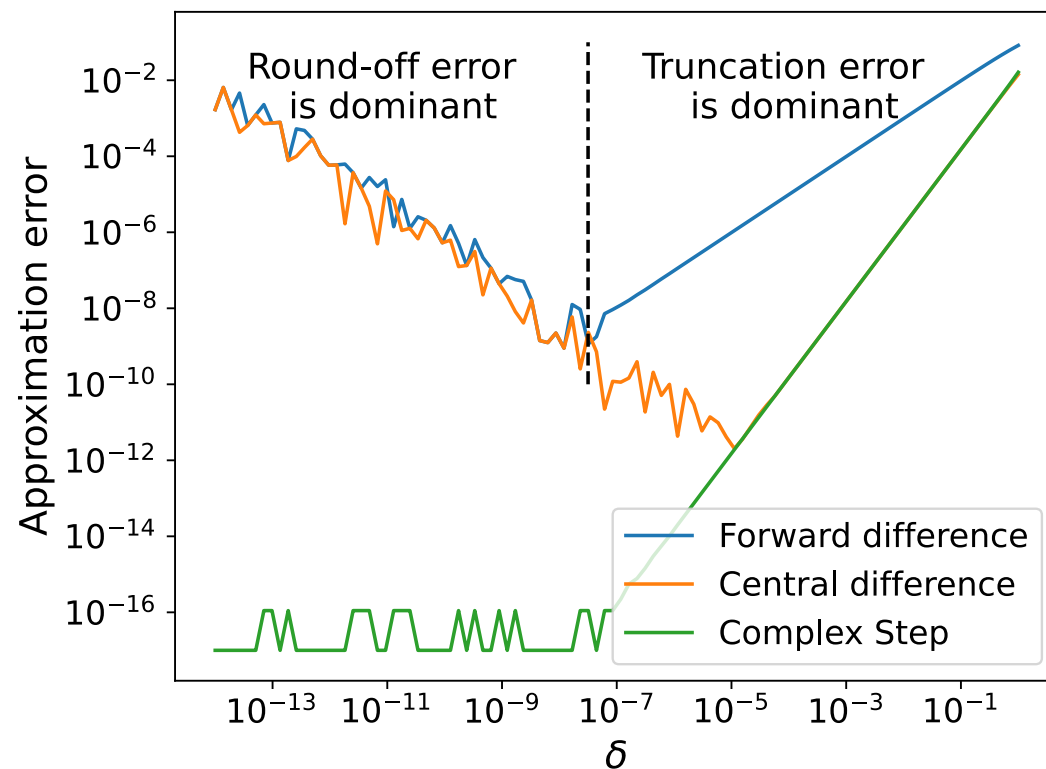
Numeric differentiation

Execute the computational graph f at nearby points (finite differences):

$$\partial f(x)[v] \approx \frac{f(x + \varepsilon v) - f(x)}{\varepsilon}$$

Great at first glance:

- Applies to arbitrary programs
- Only requires two function calls instead of one



Numerical errors:

- Taylor truncation
- Floating-point round-off

Computing a gradient is expensive: $n + 1$ evaluations

$$\nabla f(x) = \begin{pmatrix} \partial_1 f(x) \\ \partial_2 f(x) \\ \dots \\ \partial_n f(x) \end{pmatrix} \approx \frac{1}{\varepsilon} \begin{pmatrix} f(x + \varepsilon e_1) - f(x) \\ f(x + \varepsilon e_2) - f(x) \\ \vdots \\ f(x + \varepsilon e_n) - f(x) \end{pmatrix}$$

One perturbation per input dimension!

Transform the computational graph f into a new graph ∂f and propagate derivatives through it:

- Keeps the compact program encoding ($\neq$ symbolic)
- Yields exact derivative values ($\neq$ numeric)
- Can compute gradients efficiently (in reverse mode)

Important

Automatic differentiation = chain rule + layers with known derivatives.

Autodiff – modes

Derivatives as linear maps

If $f : \mathbb{R}^n \longrightarrow \mathbb{R}^m$, then $\partial f(x)$ can be represented as a Jacobian matrix:

$$J_f(x) = \left(\frac{\partial f_i}{\partial x_j}(x) \right)_{i,j} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1}(x) & \dots & \frac{\partial f_1}{\partial x_n}(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1}(x) & \dots & \frac{\partial f_m}{\partial x_n}(x) \end{pmatrix}$$

However, the linear map $v \longmapsto \partial f(x)[v]$ is natural to work with:

- generalizes to arbitrary vector spaces
- no need to materialize a matrix or flatten anything

The chain rule

Given a function composition $f = h \circ g$ with two layers, we have

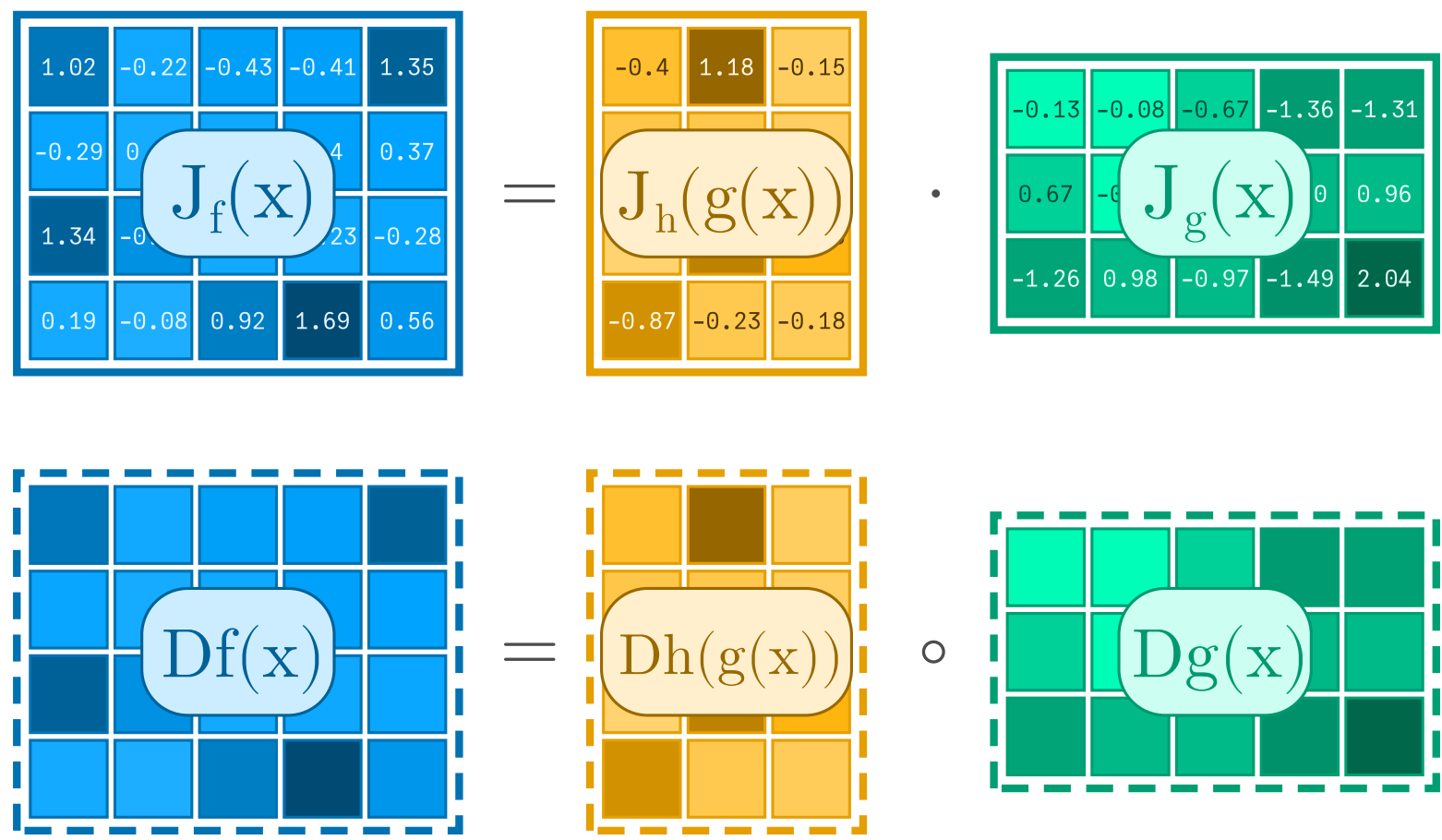
$$\partial f(x) = \partial h(g(x)) \circ \partial g(x)$$

The derivative of f is the composition of two linear maps:

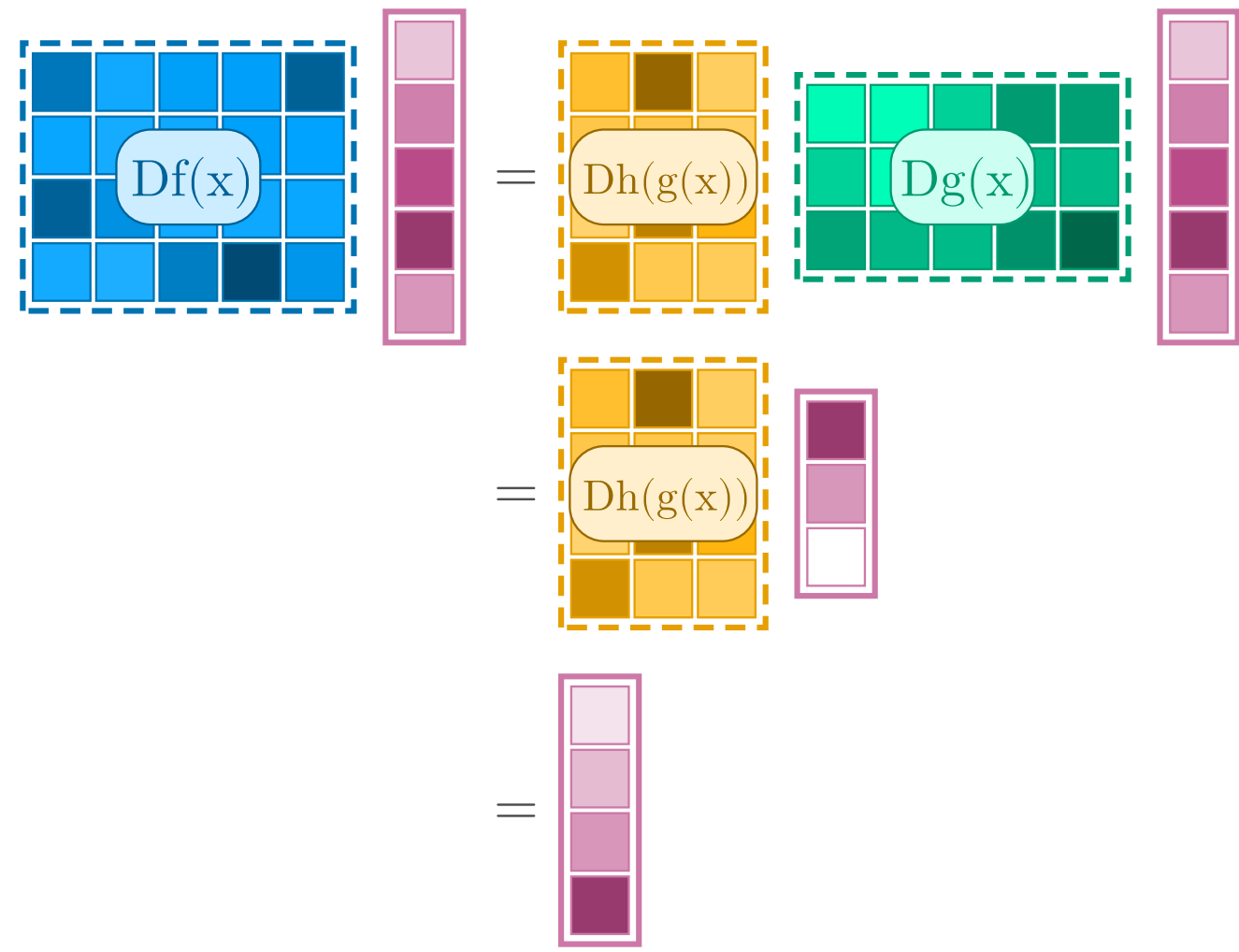
$$\partial f(x) : u \xrightarrow{\partial g(x)} v \xrightarrow{\partial h(g(x))} w$$

We can differentiate any function knowing the derivatives of its layers.

From matmul to map composition



From matmul to map composition (2)



Scalar layers

Let $\dot{x}$ denote an arbitrary input tangent (directional derivative).

variables	function f	derivative ∂f
$x \in \mathbb{R}$	ax	$a\dot{x}$
$x \in \mathbb{R}$	$\sin(x)$	$\cos(x)\dot{x}$
$x, y \in \mathbb{R}$	x, y	$x\dot{y} + y\dot{x}$

This mirrors exactly what we learned in high school.

Array layers

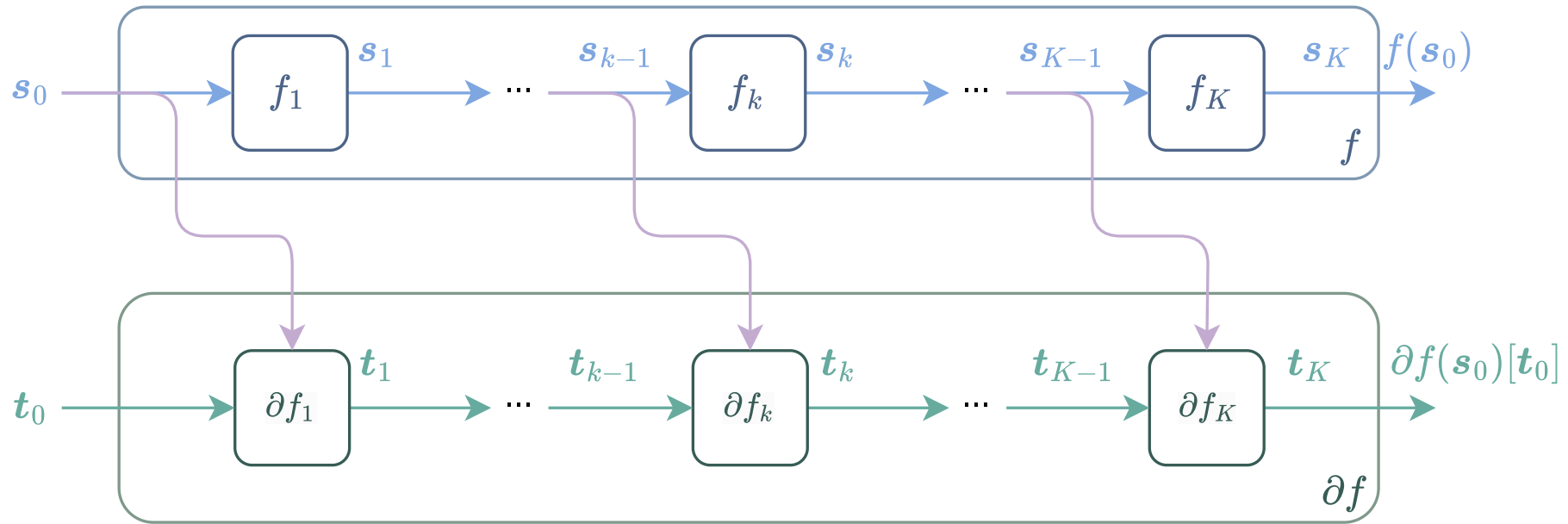
Define rules for array functions with known derivatives [5].

variables	function $f(x)$	derivative $\partial f(x)[\dot{x}]$
$x \in \mathbb{R}^n$	$Ax + b$	$A\dot{x}$
$x \in \mathbb{R}^n$	$\sigma(x)$	$\sigma'(x)\dot{x}$
$x \in \mathbb{R}^n$	$\ x\ ^2$	$2x^\top \dot{x}$
$X \in \mathbb{R}^{n \times n}$	X^{-1}	$-X^{-1} \dot{X} X^{-1}$

For the inverse, a Jacobian matrix of size $(n \times n)^2$ is not needed: the linear map is more efficient.

This was forward mode

Propagate the input and its tangent together through a chain of layers.



Back to matrices, forward mode computes JVPs $\partial f(x)[v] = J_{f(x)}v$.

With $v = e_j$, this gives a column of the Jacobian matrix:

$$\partial f(x)[e_j] = \begin{pmatrix} \frac{\partial f_1}{\partial x_1}(x) & \dots & \frac{\partial f_1}{\partial x_j}(x) & \dots & \frac{\partial f_1}{\partial x_n}(x) \\ \frac{\partial f_2}{\partial x_1}(x) & \dots & \frac{\partial f_2}{\partial x_j}(x) & \dots & \frac{\partial f_2}{\partial x_n}(x) \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1}(x) & \dots & \frac{\partial f_m}{\partial x_j}(x) & \dots & \frac{\partial f_m}{\partial x_n}(x) \end{pmatrix}$$

A tale of columns and rows

For a scalar-valued function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the Jacobian has one row:

$$J_f(x) = \nabla f(x)^\top = \left(\frac{\partial f}{\partial x_1}(x) \quad \frac{\partial f}{\partial x_2}(x) \quad \dots \quad \frac{\partial f}{\partial x_n}(x) \right)$$

We need n forward-mode JVPs to compute it column by column:

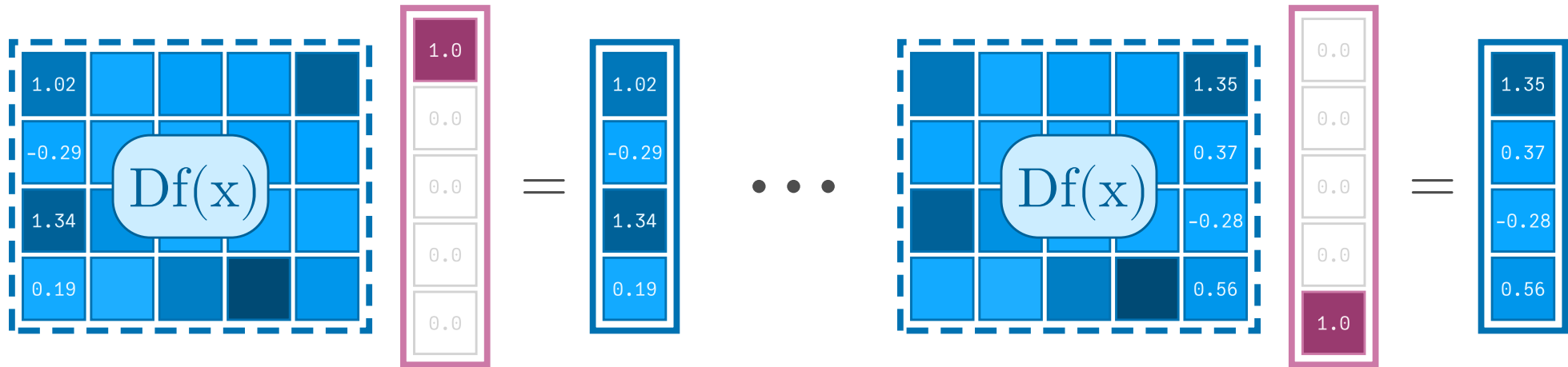
$$\partial f(x)[e_1] = \left(\frac{\partial f}{\partial x_1}(x) \quad \frac{\partial f}{\partial x_2}(x) \quad \dots \quad \frac{\partial f}{\partial x_n}(x) \right)$$

$$\partial f(x)[e_2] = \left(\frac{\partial f}{\partial x_1}(x) \quad \frac{\partial f}{\partial x_2}(x) \quad \dots \quad \frac{\partial f}{\partial x_n}(x) \right)$$

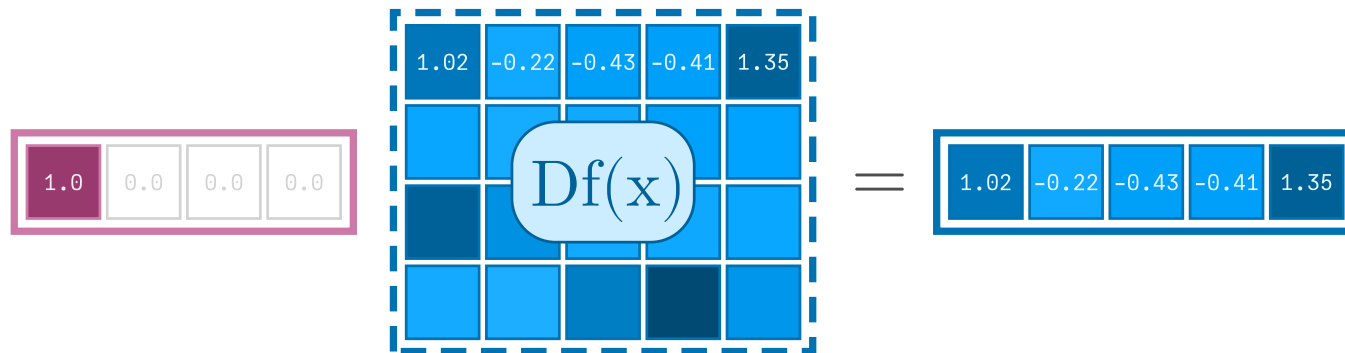
Can we compute it row by row instead, in one shot?

Columnwise Jacobian

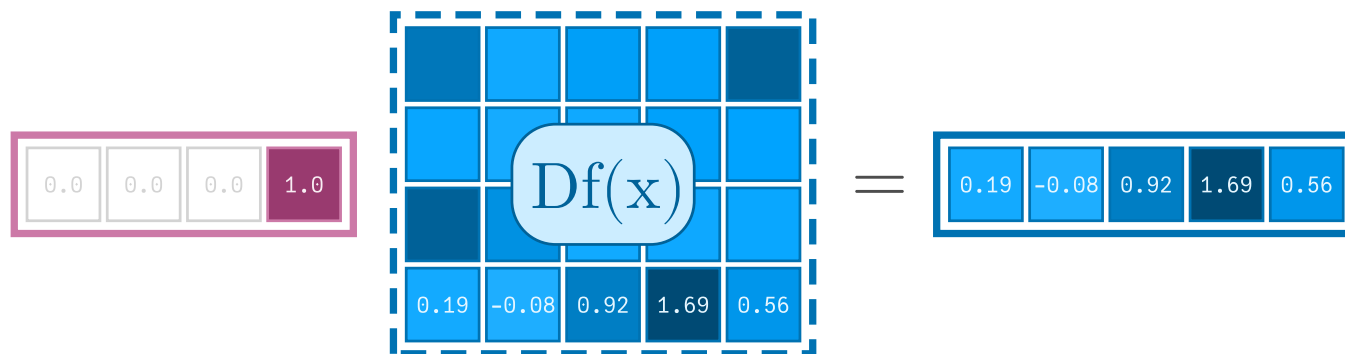
Autodiff – modes



Bad for gradients (just one row)



...



Good for gradients
(just one row)

Transpositions and adjoints

The adjoint of a linear map $\ell : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is the only linear map $\ell^* : \mathbb{R}^m \rightarrow \mathbb{R}^n$ such that

$$\forall (x, y) \in \mathbb{R}^n \times \mathbb{R}^m, \quad \langle \ell(x), y \rangle = \langle x, \ell^*(y) \rangle$$

If ℓ is represented by a matrix A , then ℓ^* is represented by $A^\top$.

If we could compute $\partial f(x)^*$, it would give us rows of the Jacobian through VJPs:

$$\partial f(x)^*[e_i] = J_{f(x)}^\top e_i = \begin{pmatrix} \frac{\partial f_1}{\partial x_1}(x) & \dots & \frac{\partial f_1}{\partial x_n}(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial f_i}{\partial x_1}(x) & \dots & \frac{\partial f_i}{\partial x_n}(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1}(x) & \dots & \frac{\partial f_m}{\partial x_n}(x) \end{pmatrix}$$

In particular, the gradient is just $\nabla f(x) = \partial f(x)^*[1]$.

Adjoint chain rule

Adjoint of linear maps reverse order, like matrix transposes:

$$\begin{aligned}\partial f(x) &= \partial h(g(x)) \circ \partial g(x) \\ \partial f(x)^* &= \partial g(x)^* \circ \partial h(g(x))^*\end{aligned}$$

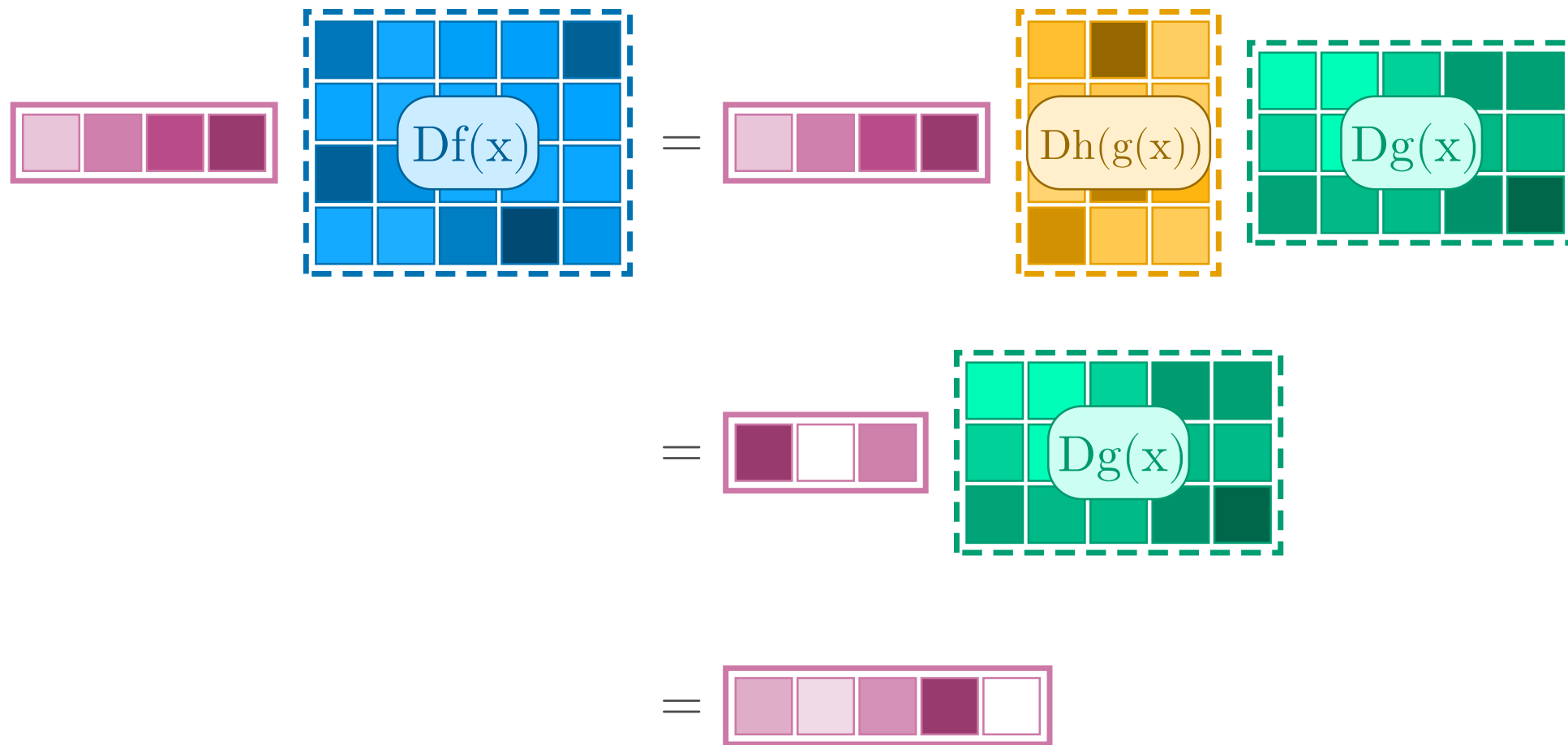
Now the propagation happens from the output back to the input:

$$\partial f(x)^* : u \xleftarrow{\partial g(x)^*} v \xleftarrow{\partial h(g(x))^*} w$$

We can differentiate any function in reverse mode knowing the adjoint derivatives of its layers.

Transposed matmul & adjoint composition

Autodiff – modes



Back to our layer examples

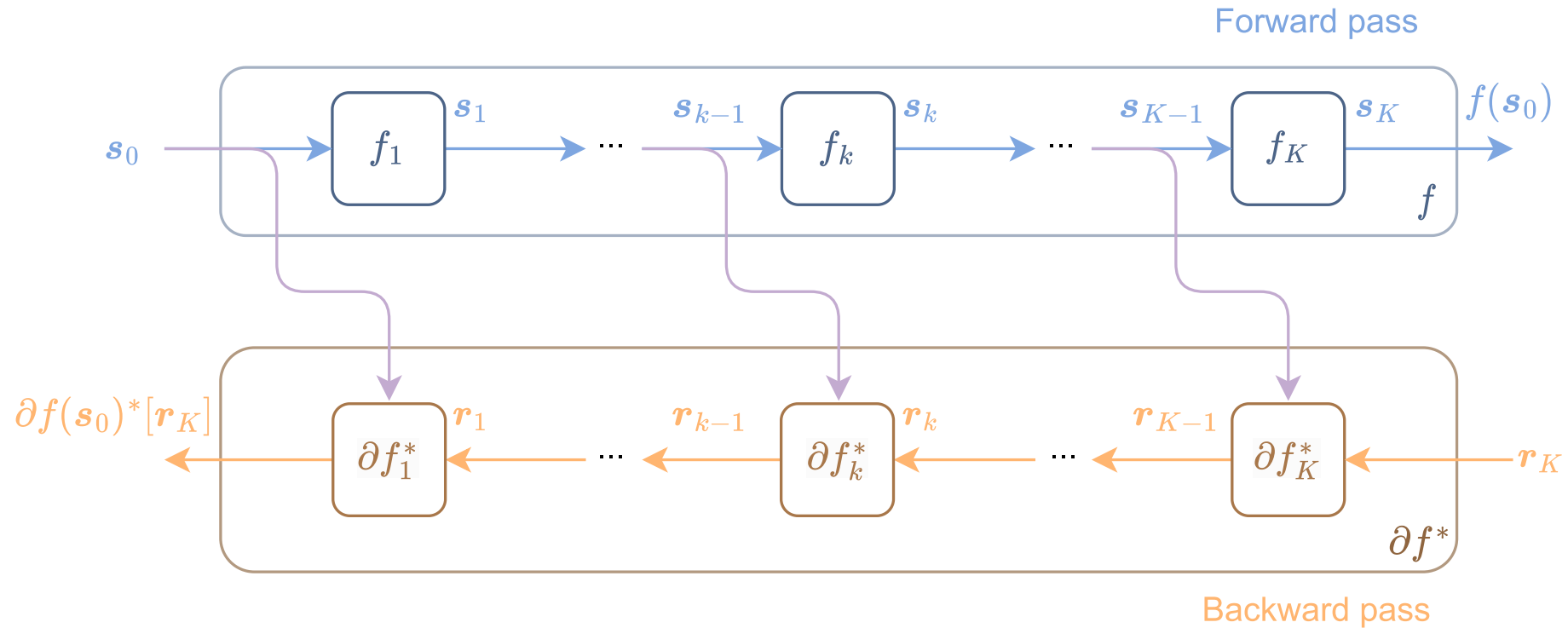
Let $\bar{y}$ be an arbitrary output cotangent (sensitivity).

variables	output	function $f(x)$	adjoint $\partial f(x)^*[\bar{y}]$
$x \in \mathbb{R}$	$y \in \mathbb{R}$	$\sin(x)$	$\cos(x)\bar{y}$
$x \in \mathbb{R}^n$	$y \in \mathbb{R}^m$	$Ax + b$	$A^\top \bar{y}$
$x \in \mathbb{R}^n$	$y \in \mathbb{R}$	$\ x\ ^2$	$2x\bar{y}$
$X \in \mathbb{R}^{n \times n}$	$Y \in \mathbb{R}^{n \times n}$	X^{-1}	$-X^{-\top} \bar{Y} X^{-\top}$

More crazy formulas in [6].

This was reverse mode

Propagate the input through a chain of layers, record enough information, backpropagate the output cotangent.



Time complexity

Theorem (Baur-Strassen):

- Cost of 1 JVP (forward mode) $\propto$ cost of 1 function call
- Cost of 1 VJP (reverse mode) $\propto$ cost of 1 function call

Rather easy to believe:

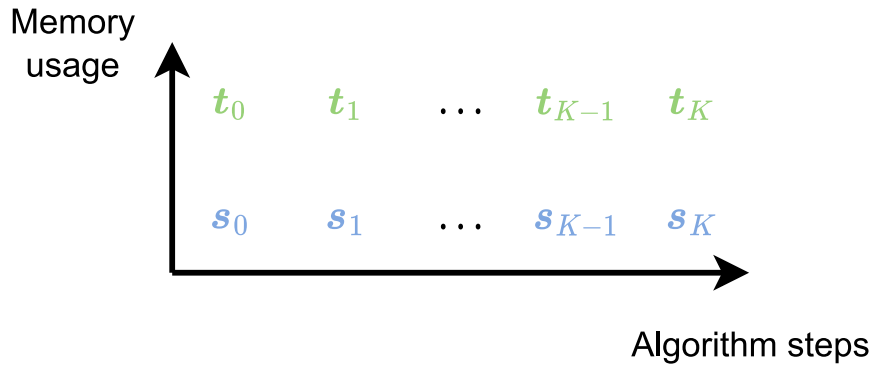
- Individual derivatives not much harder than the corresponding layer
- Composition of linear maps adds up their computational costs

Time complexity (special cases)

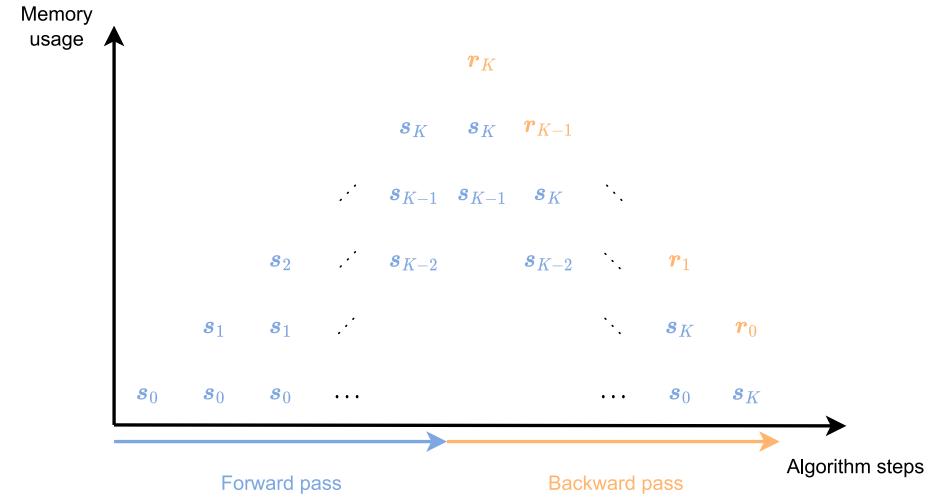
Assume the function f can be computed in time $O(\tau)$.

setting	object	forward mode	reverse mode
$\mathbb{R}^n \rightarrow \mathbb{R}^m$	Jacobian matrix	$O(n\tau)$	$O(m\tau)$
$\mathbb{R}^n \rightarrow \mathbb{R}$	gradient vector	$O(n\tau)$	$O(\tau)$
$\mathbb{R} \rightarrow \mathbb{R}^m$	derivative vector	$O(\tau)$	$O(m\tau)$

Space complexity



Forward mode has constant memory cost.



Reverse mode has linear memory cost (in the depth of the chain).

Autodiff – implementation

Autodiff reinterprets a computer program to mean something else [7].

We can take the same idea in other directions:

- Uncertainty propagation
- Physical unit checking
- Sparsity detection

Operator overloading

Pass augmented values to language operators & overload their behavior.

Example: Dual numbers in ForwardDiff.jl [8].

```
using ForwardDiff: Dual
```

```
u, ð = 2.0, 3.0
```

```
v, v̇ = 4.0, 5.0
```

```
du = Dual(u, ð)
```

```
dv = Dual(v, v̇)
```

```
julia> du * dv  
Dual{Nothing}(8.0, 22.0)
```

```
julia> u * v̇ + v * ð  
22.0
```

```
julia> du / dv  
Dual{Nothing}(0.5, 0.125)
```

```
julia> (ð * v - v̇ * u) / v^2  
0.125
```

Preprocess the source code to add derivative bookkeeping.

Example: jaxpr intermediate representation in JAX [9]

```
import jax
import jax.numpy as jnp

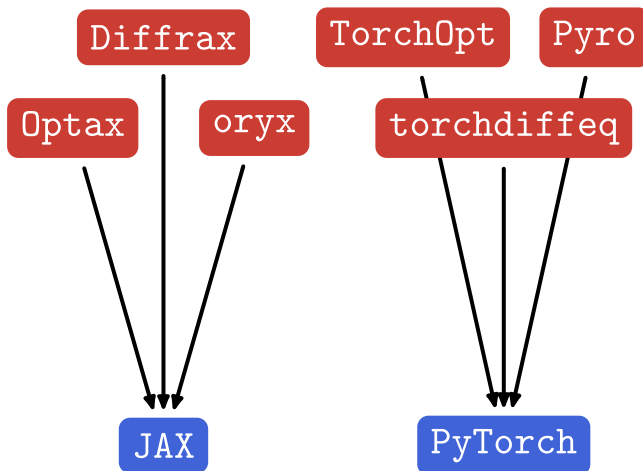
def selu(x, alpha=1.67, lambda_=1.05):
    return lambda_ * jnp.where(
        x > 0,
        x,
        alpha * jnp.exp(x) - alpha
    )

x = jnp.arange(5.0)
jax.make_jaxpr(selu)(x)
```

```
{ lambda ; a:f32[5]. let
  b:bool[5] = gt a 0.0:f32[]
  c:f32[5] = exp a
  d:f32[5] = mul 1.6699999570846558:f32[] c
  e:f32[5] = sub d 1.6699999570846558:f32[]
  f:f32[5] = jit[
    name=_where
    jaxpr={ lambda ; b:bool[5] a:f32[5]
  e:f32[5]. let
    f:f32[5] = select_n b e a
    in (f,) }
  ] b a e
  g:f32[5] = mul 1.0499999523162842:f32[] f
  in (g,) }
```

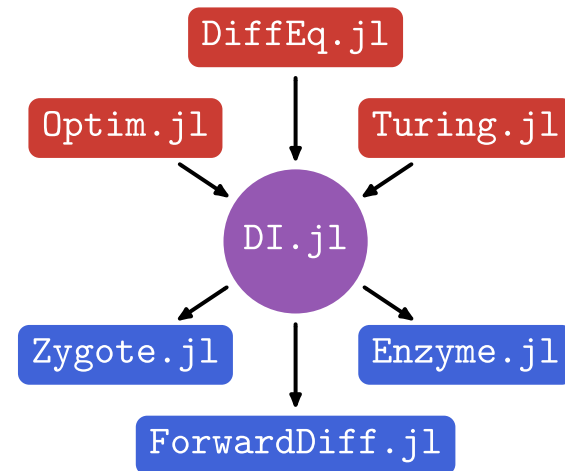
Python libraries

- [autograd](#) (historical)
- [TensorFlow](#)
- [PyTorch](#)
- [JAX](#)



Julia libraries (see [10])

- [ForwardDiff.jl](#)
- [Zygote.jl](#)
- [Enzyme.jl](#)
- [Mooncake.jl](#)



A subset of the language

Autodiff – implementation

Not every framework can differentiate every operation.

- PyTorch, JAX: restricted to domain-specific sublanguage
- Zygote.jl, JAX: error on code with mutation
- ForwardDiff.jl: errors on code with type constraints



[Open in Colab](#) [Open in Kaggle](#)

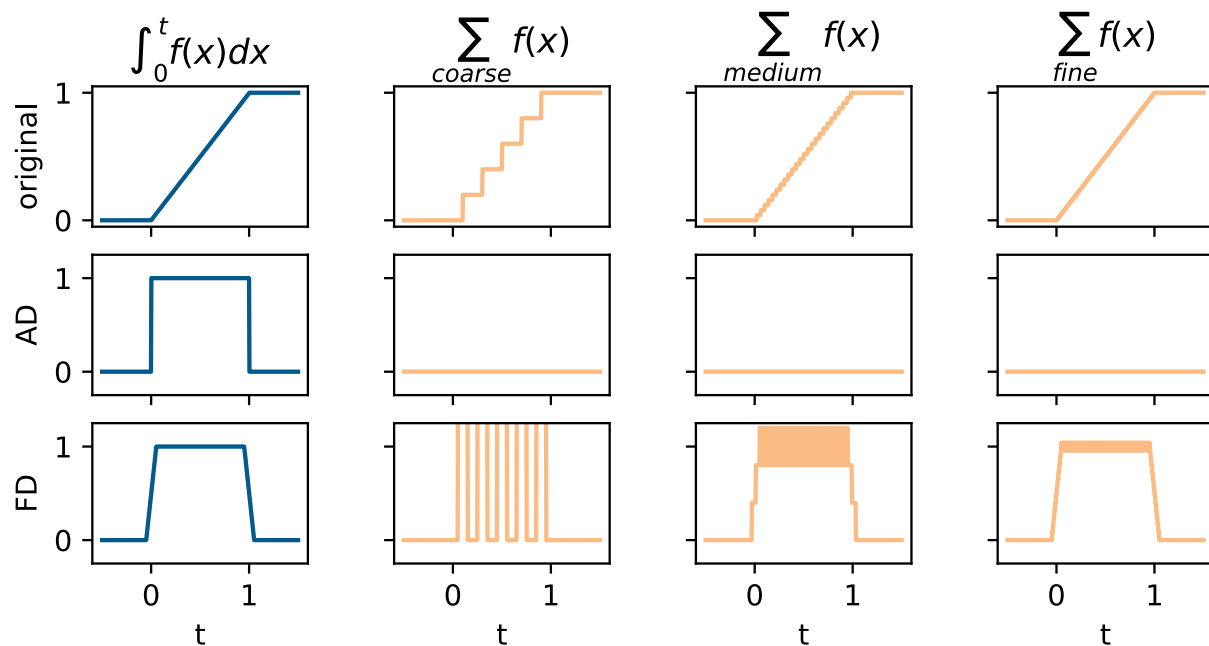
When walking about the countryside of Italy, the people will not hesitate to tell you that **JAX** has *"una anima di pura programmazione funzionale"*.

JAX is a language for **expressing** and **composing transformations** of numerical programs. **JAX** is also able to **compile** numerical programs for CPU or accelerators (GPU/TPU). JAX works great for many numerical and scientific programs, but **only if they are written with certain constraints** that we describe below.

When to stop autodiffing

A function is approximated by a program [11]. We can either

1. Differentiate the approximation
2. Approximate the derivative



Example: Heron's method

Autodiff – implementation

The following iteration approximates the square root of $a \in [0, \infty[$:

$$x_0 = a, \quad x_{n+1} = \frac{1}{2} \left(x_n + \frac{a}{x_n} \right)$$

$$x_0 = a$$

$$\dot{x}_0 = 1$$

While not converged

$$x_{n+1} = \frac{1}{2} \left(x_n + \frac{a}{x_n} \right)$$

$$\dot{x}_{n+1} = \frac{1}{2} \left(\dot{x}_n + \frac{1}{x_n} - \frac{a \dot{x}_n}{x_n^2} \right)$$

Return $\dot{x}_N$

$$x_0 = a$$

While not converged

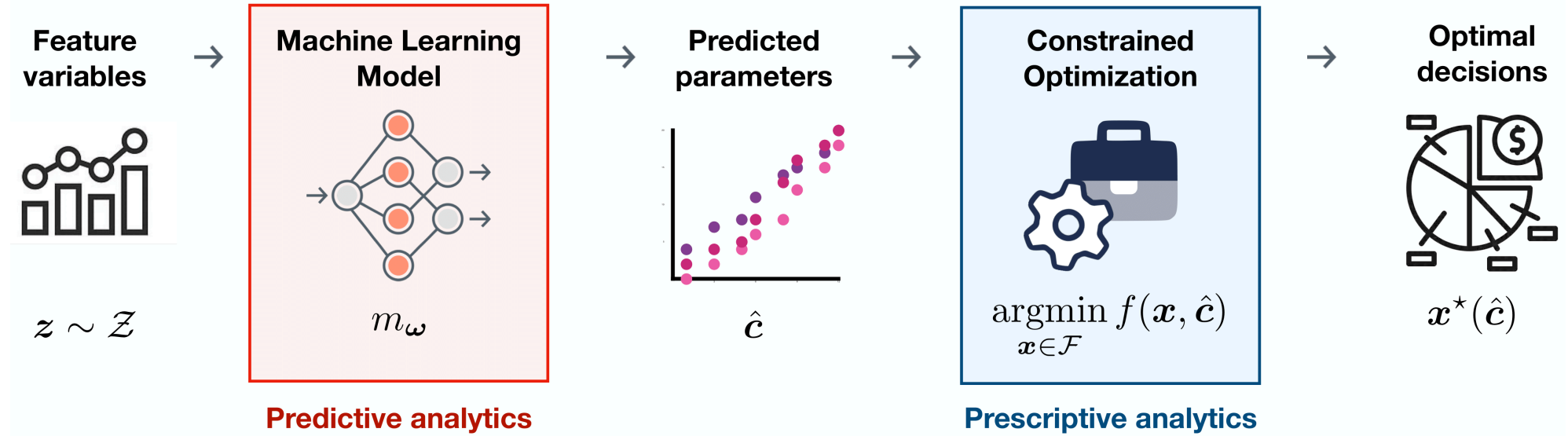
$$x_{n+1} = \frac{1}{2} \left(x_n + \frac{a}{x_n} \right)$$

Return $\frac{1}{2x_N}$

CO layers – theory

Decision-focused learning

Using operations research solvers inside learning pipelines [12].



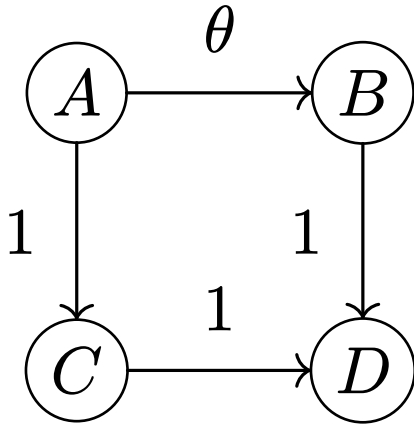
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Differentiation challenge

Discrete solvers and program elements don't have useful derivatives.

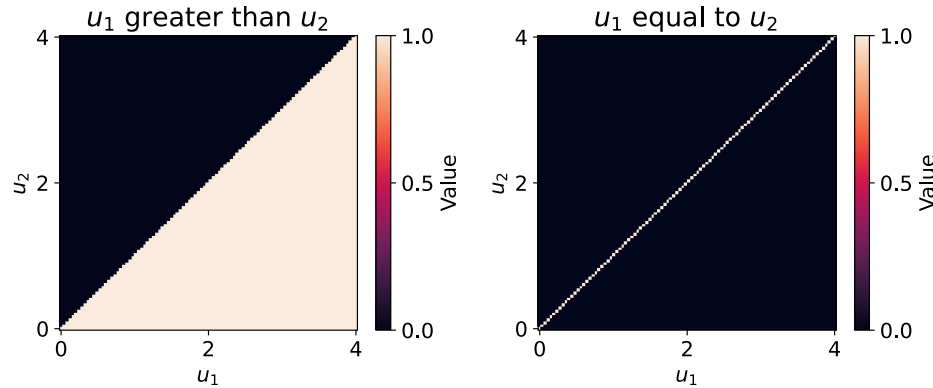


$$\text{shortest_path}(\theta) = \begin{cases} \text{ABD} & \text{if } \theta < 1 \\ \text{ACD} & \text{if } \theta > 1 \\ \text{both} & \text{if } \theta = 1 \end{cases}$$

Piecewise-constant function!

We need a nicely differentiable surrogate to allow backpropagation.

Branching

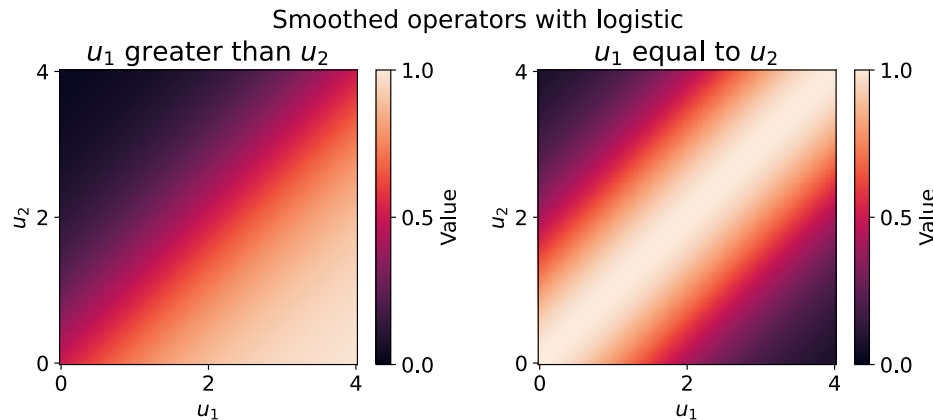


Approximate the step function

$$\text{step}(u) = \begin{cases} 1 & \text{if } u \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

with a sigmoid function

$$\text{sigmoid}_{\beta}(u) = \frac{1}{1 + e^{-\beta u}}$$



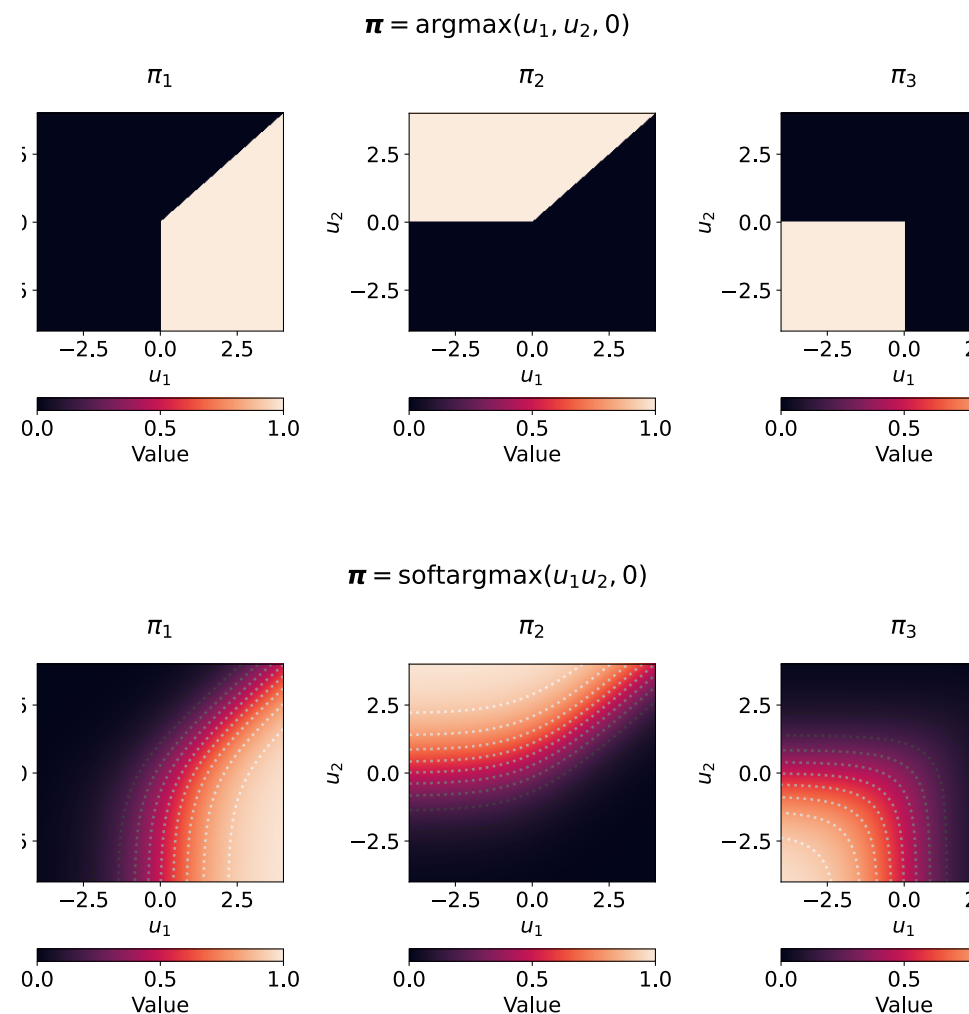
Temperature parameter $\frac{1}{\beta}$ controls smoothness and precision.

Choosing

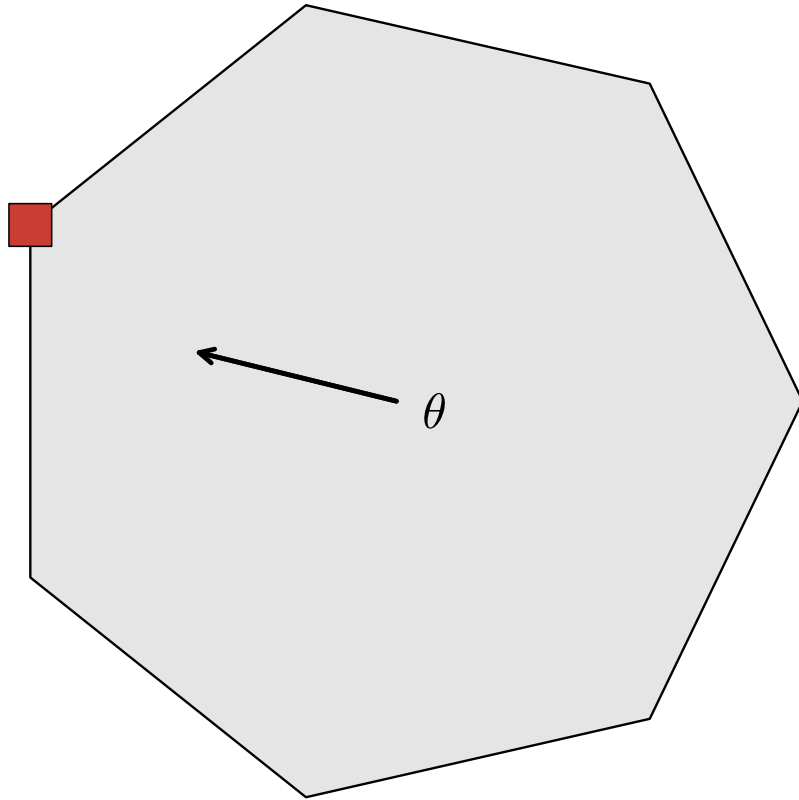
Approximate $\operatorname{argmax}(\theta)$ with

$$\operatorname{softmax}_{\beta}(\theta) = \left(\frac{\exp(\beta\theta_i)}{\sum_j \exp(\beta\theta_j)} \right)_i$$

$$\operatorname{sparsemax}(\theta) = \operatorname{argmin}_{p \in \Delta} \|p - \theta\|^2$$



Linear programs



Focus on LPs where the cost vector θ varies:

$$x^*(\theta) = \operatorname{argmax}_x \theta^\top x \text{ s.t. } Ax \leq b$$

Feasible set is a polyhedron.

Almost surely on θ , the optimum $x^*(\theta)$ is a vertex.

Requires smoothing!

CO layers – differentiation

Two paradigms

- Regularization
- Integration

CO layers – differentiation

Regularization

Solve a convex program instead to get.

Quadratic regularization [13]:

$$x_{\gamma}^{\star}(\theta) = \operatorname{argmax}_x \theta^{\top} x - \gamma \|x\|^2 \text{ s.t. } Ax \leq b$$

Logarithmic barrier [14]:

$$x_{\gamma}^{\star}(\theta) = \operatorname{argmax}_x \theta^{\top} x - \gamma \sum_i \log(s_i) \text{ s.t. } Ax + s = b$$

Must use a different solver.

Suppose we know optimality conditions that are satisfied by $x^*(\theta)$:

$$c(x^*(\theta), \theta) = 0$$

Differentiating through those conditions yields:

$$\underbrace{\partial_x c}_A \underbrace{\partial_\theta x^*}_J + \underbrace{\partial_\theta c}_{-B} = 0$$

By solving the linear system $AJ = B$, we recover the Jacobian $\partial_\theta x^*(\theta)$.

Hypotheses: smoothness, A must be invertible.

Optimization (and other) layers are defined by what they return, not how they compute it.

The implicit function theorem can be inserted into autodiff [15].

$$AJ = B \quad \text{with } A = \partial_x c, B = -\partial_\theta c, J = \partial_\theta x^\star$$

Forward mode

$$A(Jv) = Bv$$

1. Compute $w = Bv$
2. Solve $Au = w$ for u

Reverse mode

$$v^\top J = u^\top AJ = u^\top B$$

1. Solve $A^\top u = v$ for u
2. Compute $u^\top B$

OptNet, the grandfather of optimization layers [16]

$$\underbrace{x^*(Q, q, A, b, G, h)}_{\theta} = \operatorname{argmin}_x \frac{1}{2} x^\top Q x + q^\top x \quad \text{s.t.} \quad Ax = b, Gx \leq h$$

With dual variables ν and $\lambda \geq 0$, the Lagrangian is:

$$\mathcal{L}(x, \nu, \lambda) = \frac{1}{2} x^\top Q x + q^\top x + \nu^\top (Ax - b) + \lambda^\top (Gx - h)$$

The KKT conditions (without inequalities) are:

$$Qx^* + q + A^\top \nu^* + G^\top \lambda^* = 0$$

$$Ax^* - b = 0$$

$$\text{diag}(\lambda^*)(Gx^* - h) = 0$$

And we apply the implicit function theorem:

$$\frac{d}{d\theta} c((x^*(\theta), \nu^*(\theta), \lambda^*(\theta)), \theta) = 0$$

Requires the dual optimal solution as well.

Picking optimality conditions

CO layers – differentiation

Related to the algorithm used for $\operatorname{argmax}_{x \in \mathcal{X}} f(x, \theta)$.

Constraints	Algo	Conditions
$\mathcal{X} = \mathbb{R}^n$	Gradient descent	$\nabla_x f(x, \theta) = 0$
$\mathcal{X} = \begin{cases} G(x, \theta) \leq 0 \\ H(x, \theta) = 0 \end{cases}$	Primal-dual	Karush-Kuhn-Tucker
$\mathcal{X}$ projection-friendly	Projected gradient descent	$\operatorname{proj}_{\mathcal{X}}(x - \eta \nabla_x f(x, \theta)) = x$
$\mathcal{X}$ LP-friendly	Frank-Wolfe	Projected GD on the simplex

More examples in [15].

Probabilistic interpretation

CO layers – differentiation

Instead of picking a single polytope vertex, construct a probability distribution over solutions [17], [18], [19].

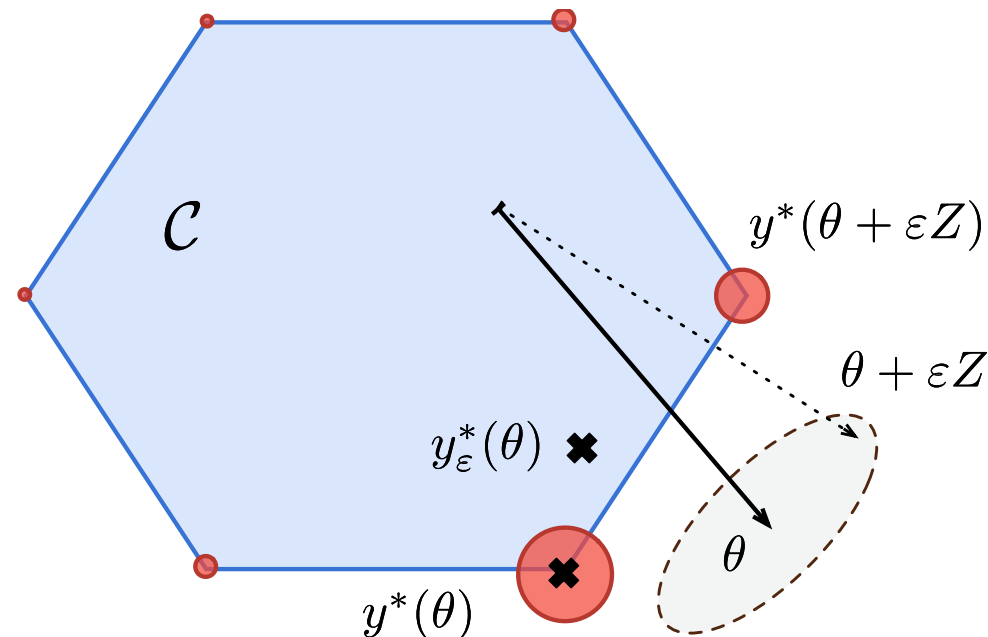
Generalizes beyond optimality: [20] studies the case of a local search, framed as MCMC simulation.

Perturbation

Use REINFORCE gradient on the perturbed problem [21]:

$$x_{\varepsilon}^*(\theta) = \mathbb{E} \left[\operatorname{argmax}_{x \in \mathcal{C}} (\theta + \varepsilon Z)^{\top} x \right]$$

Other perturbations are possible to ensure positivity [17].



Consider the parametric expectation / integral [22]

$$f(\theta) = \mathbb{E}_{X \sim p(\theta)}[g(X)] = \int g(x)p(x, \theta) \, dx$$

approximated by the Monte-Carlo estimate

$$f_N(\theta) = \frac{1}{N} \sum_{n=1}^N g(x^{(n)}) \quad \text{where} \quad x^{(n)} \sim p(\theta)$$

Can we find a Monte-Carlo estimate for $\nabla f(\theta)$ too? Not obvious:

$$\nabla f(\theta) = \int g(x) \nabla_{\theta} p(x, \theta) \, dx$$

The score function is defined by

$$\nabla_{\theta} \log p(x, \theta) = \frac{\nabla_{\theta} p(x, \theta)}{p(x, \theta)}$$

We use it as follows:

$$\begin{aligned} \nabla f(\theta) &= \int g(x) \nabla_{\theta} p(x, \theta) \, dx = \int g(x) \frac{\nabla_{\theta} p(x, \theta)}{p(x, \theta)} p(x, \theta) \, dx \\ &= \int [g(x) \nabla_{\theta} \log p(x, \theta)] p(x, \theta) \, dx \\ &= \mathbb{E}_{X \sim p(\theta)} [g(X) \nabla_{\theta} \log p(X, \theta)] \end{aligned}$$

We approximate the gradient

$$\nabla f(\theta) = \nabla_{\theta} \mathbb{E}_{X \sim p(\theta)} [g(X)] = \mathbb{E}_{X \sim p(\theta)} [g(X) \nabla_{\theta} \log p(X, \theta)]$$

with Monte-Carlo using the same distribution $p(\theta)$

$$\frac{1}{N} \sum_{n=1}^N g(x^{(n)}) \nabla_{\theta} \log p(x^{(n)}, \theta)$$

Also known as REINFORCE gradient.

Assume we can rewrite $X \sim p(\theta)$ as $X = h(\theta, Z)$ with $Z \sim q$ independent from θ (decouple sampling and transformation):

$$\mathbb{E}_{X \sim p(\theta)}[g(X)] = \mathbb{E}_{Z \sim q}[g(h(\theta, Z))]$$

$$\nabla_{\theta} \mathbb{E}_{X \sim p(\theta)}[g(X)] = \mathbb{E}_{Z \sim q}[\nabla_{\theta}(g \circ h)(\theta, Z)]$$

We approximate with Monte-Carlo using a different distribution q :

$$\frac{1}{N} \sum_{n=1}^N \nabla_{\theta}(g \circ h)(\theta, z^{(n)}) \quad \text{where} \quad z^{(n)} \sim q$$

Also known as the reparametrization trick.

- Gaussian: $X \sim \mathcal{N}(\mu, \Sigma)$ yields

$$X = \mu + LZ \quad \text{with} \quad Z \sim \mathcal{N}(0, I) \quad \text{and} \quad LL^\top = \Sigma$$

- Same idea for other location-scale distributions
- Invert the cumulative distribution function:

$$X = F^{-1}(U) \quad \text{where} \quad F(x) = \mathbb{P}(X \leq x) \quad \text{and} \quad U \sim \mathcal{U}(0, 1)$$

	Score function	Pathwise
Unbiased	yes	yes
Consistent	yes	yes
Hypotheses on p	smooth	reparametrizable
Hypotheses on g	none	smooth
Variance	high	low

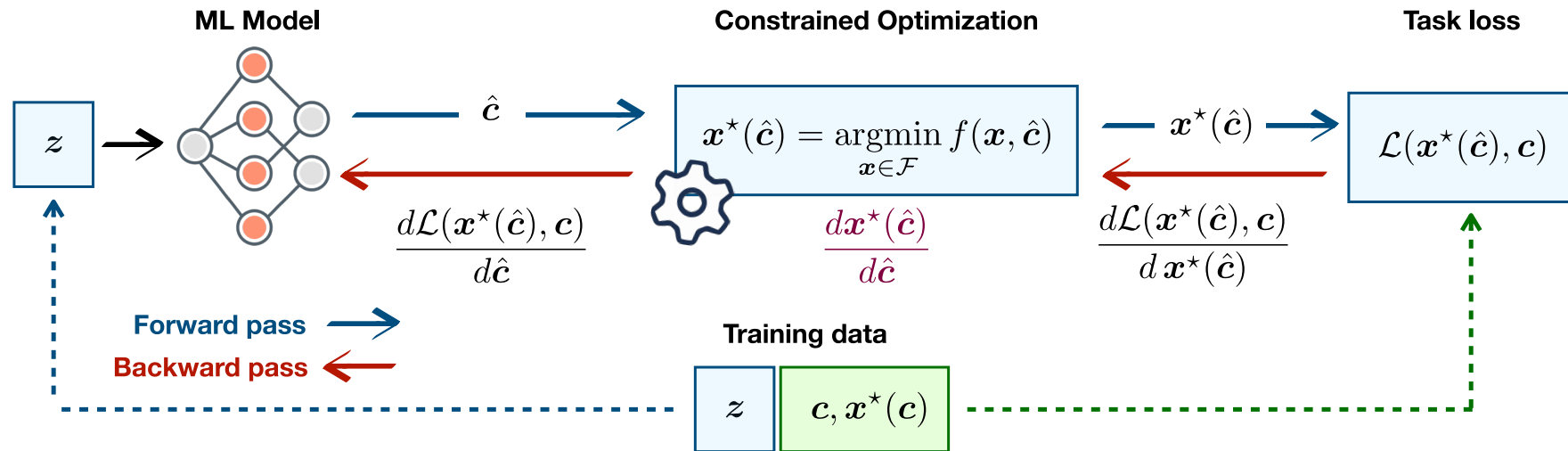
Score function is more widely applicable (black box g , discrete p).

Pathwise is more robust (lower variance, stable with dimension).

Backpropagating through the solver itself may not be necessary.

Some loss functions provide (sometimes convex) surrogates [12]:

- SPO+ loss [23]
- Geometric losses [25], [26]
- Fenchel-Young loss [24]
- Local losses [27], [28]



CO layers – practice

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

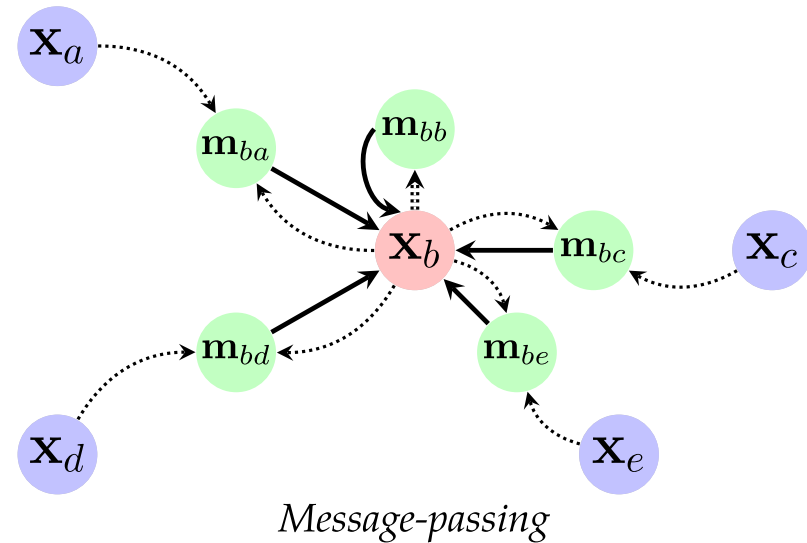
Graph Neural Networks

Graph-equivariant functions f are defined on node features X such that

$$f(PX) = Pf(X) \text{ for all permutations } P$$

Message-passing neural networks [29] are one such family:

$$h_u = \varphi \left(x_u, \bigoplus_{v \in \mathcal{N}_u} \psi(x_u, x_v) \right)$$



Example: learning to rank

CO layers – practice

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Example: shortest paths on a map

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Example: stochastic vehicle scheduling

CO layers – practice

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Example: two-stage maximum spanning

CO layers – practice

trees Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Python libraries

- `cvxpylayers` [30]
- `TorchOpt` [31]
- `optax` [32]
- `Theseus` [33]
- `PyEP0` [34]

Julia libraries

- `DiffOpt.jl` [35]
- `ImplicitDifferentiation.jl` [36]
- `InferOpt.jl` [17]

- [1] M. Blondel and V. Roulet, “The Elements of Differentiable Programming.” [Online]. Available: <http://arxiv.org/abs/2403.14606>
- [2] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html
- [3] S. Scardapane, “Alice’s Adventures in a Differentiable Wonderland – Volume I, A Tour of the Land.” [Online]. Available: <http://arxiv.org/abs/2404.17625>
- [4] S. Laue, “On the Equivalence of Automatic and Symbolic Differentiation.” [Online]. Available: <http://arxiv.org/abs/1904.02990>
- [5] K. B. Petersen and M. S. Pedersen, “The Matrix Cookbook,” 2012. [Online]. Available: <https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf>
- [6] M. Giles, “An Extended Collection of Matrix Derivative Results for Forward and Reverse Mode Automatic Differentiation,” 2008, [Online]. Available: <https://ora.ox.ac.uk/objects/uuid:8d0c0a29-c92b-4153-a1d2-38b276e93124>
- [7] C. C. Margossian, “A Review of Automatic Differentiation and Its Efficient Implementation,” *WIREs Data Mining and Knowledge Discovery*, vol. 9, no. 4, p. e1305, 2019, doi: [10.1002/widm.1305](https://doi.org/10.1002/widm.1305).
- [8] J. Revels, M. Lubin, and T. Papamarkou, “Forward-Mode Automatic Differentiation in Julia.” [Online]. Available: <http://arxiv.org/abs/1607.07892>
- [9] J. Bradbury *et al.*, “JAX: Composable Transformations of Python+NumPy Programs.” [Online]. Available: <http://github.com/google/jax>
- [10] G. Dalle and A. Hill, “A Common Interface for Automatic Differentiation.” [Online]. Available: <http://arxiv.org/abs/2505.05542>
- [11] J. Hückelheim, H. Menon, W. Moses, B. Christianson, P. Hovland, and L. Hascoët, “A Taxonomy of Automatic Differentiation Pitfalls,” *WIREs Data Mining and Knowledge Discovery*, vol. 14, no. 6, p. e1555, 2024, doi: [10.1002/widm.1555](https://doi.org/10.1002/widm.1555).
- [12] J. Mandi *et al.*, “Decision-Focused Learning: Foundations, State of the Art, Benchmark and Future Opportunities,” *Journal of Artificial Intelligence Research*, vol. 80, pp. 1623–1701, Aug. 2024, doi: [10.1613/jair.1.15320](https://doi.org/10.1613/jair.1.15320).

- [13] B. Wilder, B. Dilkina, and M. Tambe, “Melding the Data-Decisions Pipeline: Decision-Focused Learning for Combinatorial Optimization,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, pp. 1658–1665, July 2019, doi: [10.1609/aaai.v33i01.33011658](https://doi.org/10.1609/aaai.v33i01.33011658).
- [14] J. Mandi and T. Guns, “Interior Point Solving for LP-based Prediction+optimisation,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2020, pp. 7272–7282. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/51311013e51adebc3c34d2cc591fefee-Abstract.html>
- [15] M. Blondel *et al.*, “Efficient and Modular Implicit Differentiation,” in *Advances in Neural Information Processing Systems*, Dec. 2022, pp. 5230–5242. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/228b9279ecf9bbafe582406850c57115-Abstract-Conference.html
- [16] B. Amos and J. Z. Kolter, “OptNet: Differentiable Optimization as a Layer in Neural Networks,” in *Proceedings of the 34th International Conference on Machine Learning*, PMLR, July 2017, pp. 136–145. [Online]. Available: <https://proceedings.mlr.press/v70/amos17a.html>
- [17] G. Dalle, L. Baty, L. Bouvier, and A. Parmentier, “Learning with Combinatorial Optimization Layers: A Probabilistic Approach.” [Online]. Available: <http://arxiv.org/abs/2207.13513>
- [18] M. Niepert, P. Minervini, and L. Franceschi, “Implicit MLE: Backpropagating Through Discrete Exponential Family Distributions,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2021, pp. 14567–14579. [Online]. Available: <https://proceedings.neurips.cc/paper/2021/hash/7a430339c10c642c4b2251756fd1b484-Abstract.html>
- [19] M. Paulus, D. Choi, D. Tarlow, A. Krause, and C. J. Maddison, “Gradient Estimation with Stochastic Softmax Tricks,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2020, pp. 5691–5704. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/3df80af53dce8435cf9ad6c3e7a403fd-Abstract.html>
- [20] G. Vivier-Ardisson, M. Blondel, and A. Parmentier, “Learning with Local Search MCMC Layers.” [Online]. Available: <http://arxiv.org/abs/2505.14240>
- [21] Q. Berthet, M. Blondel, O. Teboul, M. Cuturi, J.-P. Vert, and F. Bach, “Learning with Differentiable Perturbed Optimizers,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2020, pp. 9508–9519. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/6bb56208f672af0dd65451f869fedfd9-Abstract.html>

- [22] S. Mohamed, M. Rosca, M. Figurnov, and A. Mnih, “Monte Carlo Gradient Estimation in Machine Learning,” *Journal of Machine Learning Research*, vol. 21, no. 132, pp. 1–62, 2020, [Online]. Available: <http://jmlr.org/papers/v21/19-346.html>
- [23] A. N. Elmachtoub and P. Grigas, “Smart ``Predict, Then Optimize”,” *Management Science*, vol. 68, no. 1, pp. 9–26, Jan. 2022, doi: [10.1287/mnsc.2020.3922](https://doi.org/10.1287/mnsc.2020.3922).
- [24] M. Blondel, A. F. T. Martins, and V. Niculae, “Learning with Fenchel-Young Losses,” *Journal of Machine Learning Research*, vol. 21, no. 35, pp. 1–69, 2020, [Online]. Available: <http://jmlr.org/papers/v21/19-021.html>
- [25] B. Tang and E. B. Khalil, “CaVE: A Cone-Aligned Approach for~Fast Predict-then-optimize with~Binary Linear Programs,” in *Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, B. Dilkina, Ed., Cham: Springer Nature Switzerland, 2024, pp. 193–210. doi: [10.1007/978-3-031-60599-4_12](https://doi.org/10.1007/978-3-031-60599-4_12).
- [26] S. Berden, A. İ. Mahmutoğulları, D. Tsouros, and T. Guns, “Solver-Free Decision-Focused Learning for Linear Optimization Problems.” [Online]. Available: <http://arxiv.org/abs/2505.22224>
- [27] A. Zharmagambetov, B. Amos, A. Ferber, T. Huang, B. Dilkina, and Y. Tian, “Landscape Surrogate: Learning Decision Losses for Mathematical Optimization Under Partial Information,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 27332–27350, Dec. 2023, [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/574f145eac328cc4aaf9358e27120eb5-Abstract-Conference.html
- [28] S. Shah, K. Wang, B. Wilder, A. Perrault, and M. Tambe, “Decision-Focused Learning without Decision-Making: Learning Locally Optimized Decision Losses,” in *Advances in Neural Information Processing Systems*, Dec. 2022, pp. 1320–1332. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/0904c7edde20d7134a77fc7f9cd86ea2-Abstract-Conference.html
- [29] P. Veličković, “Message Passing All the Way Up.” [Online]. Available: <http://arxiv.org/abs/2202.11097>
- [30] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and J. Z. Kolter, “Differentiable Convex Optimization Layers,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2019. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/9ce3c52fc54362e22053399d3181c638-Abstract.html>

- [31] J. Ren *et al.*, “TorchOpt: An Efficient Library for Differentiable Optimization,” *Journal of Machine Learning Research*, vol. 24, no. 367, pp. 1–14, 2023, [Online]. Available: <http://jmlr.org/papers/v24/23-0191.html>
- [32] DeepMind *et al.*, “The DeepMind JAX Ecosystem.” [Online]. Available: <http://github.com/google-deepmind>
- [33] L. Pineda *et al.*, “Theseus: A Library for Differentiable Nonlinear Optimization.” [Online]. Available: <http://arxiv.org/abs/2207.09442>
- [34] B. Tang and E. B. Khalil, “PyEPO: A PyTorch-based End-to-End Predict-Then-Optimize Library for Linear and Integer Programming,” *Mathematical Programming Computation*, vol. 16, no. 3, pp. 297–335, Sept. 2024, doi: [10.1007/s12532-024-00255-x](https://doi.org/10.1007/s12532-024-00255-x).
- [35] A. Sharma, M. Besançon, J. D. Garcia, and B. Legat, “Flexible Differentiable Optimization via Model Transformations.” [Online]. Available: <http://arxiv.org/abs/2206.06135>
- [36] G. Dalle, “Machine Learning and Combinatorial Optimization Algorithms, with Applications to Railway Planning,” Doctoral dissertation, 2022. [Online]. Available: <https://pastel.hal.science/tel-04053322>